

LLM: Transformer

Transformer jarayonida 3 ta step bor.

1. Block foundations
2. transformer architecture
3. training & performance

Block Foundations

Block Foundations:

- Layers
- Normalization
- Activation Functions

Shu 3 ta component dan iborat

Block Foundations: Layers

Layers: input qiymatni output gacha chiqarish uchun kerak bulodigon matematik Formulalar ketma-ketligi. Weightlar yordamida utkizadi

Sub-layer: Layerlar asosan 2 qismdan tashkil topgan va har birini uzini vazifasi bor

1. **Attention layer:** tokenlar bir-biri bilan gaplashib orasidagi bogliqlikni hisoblaydi. Context Engine. Manosini eslab qoladi
2. **Feed-Forward Network layer:** suzlar alohida ishlashini taminlaydi. Knowledge Engine

Feature	Attention layer (muloqot & Context)	FFN Layer (bilim & factlar library)
Asosiy task	Tokenlarni bogliqligi	Har bir tokenni alohida qayta ishlaydi
Ishlashi	Global	Local
GPU mathematic	Ketma-ketlikka bogliq	Ulchamga bogliq
Nimani urganadi	Syntaxis, grammatika	Dunyoqarash, aniq factlar, logic
Modern techs	Grouped-Query Attention	SwiGLU

Layerlar architecturasi:

1. Tokenization and Embeddings: tokenlarga ajratish va low-dimensionalga utkazish
2. Attention layers: token orasidagi bogliqliklarni hisoblash
3. FFN layers: faqtni qidirish. Har token alohida ishlanib chiqadi
4. javobni tayyorlab birlashtirish

Layerlarni joylashtiruv usullari bor:

1. **Serial**

Data → Attention → FFN

2. **Parallel**

Data → Attention & FFN → Birlashtirish

Bu terminlar SML da ham bulgan. Ensemble algorithm methodida Stacking – serial yani sequence holatda ishlagan. A Bagging – parallel holda ishlagan.

Compare	Serial Architecture	Parallel Architecture
Mathematic oqim	Data → attention → FFN	Attention, FFN layerlar teng ishlaydi
GPU tezligi	Sekinroq	~15% tezroq
Accuracy	Standard va barqaror	Serial bilan birhil
Plus	Code qismi oson Transformer qoidalari mos	Compute saving, modern GPU lar uchun ideal mos
Minus	Katta modellar juda qimmatga tushadi, sekin	Code, matrixlar qiyinroq
Usage case	Kichik experiment	Modern LLM, katta joylarda

Block Foundations: Normalization

Normalization: NN ichidagi raqamlarni kichik holatda ushlab turish. Oldin utkan scaling mavzusi bilan birhil

Normalizationni 2ta turi bor:

1. **LayerNorm** (GPT2, GPT3):

- Eski LLM lar ishlatadigan usul
- Mean yoki Variance ga almashtiradi

- Oddiy Mean va Variance ni topish uchun hardoim GPU qushimcha memory ishlatadi, va bu ortiqcha harajat hisoblanadi

2. **RMSNorm** (Root Mean Square Normalization):

- Modern usul
- Butun ustunni emas, har bir suzni uzini kvadratik(RMSE) iga buladi
- GPU ozroq ishlatiladi

Feature	LayerNorm (Classic)	RMSNorm (Modern)
Hisoblash usuli	Mean & Varianceni hisoblaydi	Faqat kvadratik (RMS) qiymatni hisoblaydi
GPU tezligi	Standart tezlikda ishlaydi (Sekinroq)	10% ~ 50% gacha tezroq (Math amallar ozligi uchun)
Model aniqligi	Barqaror va yaxshi	LayerNorm bilan bir hil, lekin resource juda tejaydi
Where use	Eski GPT modellarida, BERT va original Transformerda	LLaMA 3, GPT-4, Claude 3, Gemini Va barcha yangi LLM larda

Block Foundations: Activation Functions

Activation Function: NN ga non-linearity qushish. Reasoning ham qushiladi

Turi	Formula	LLM dagi holati	Kimlar ishlatadi?
ReLu	$\text{Max}(0, x)$	Eskirgan. Neyronlarni tashavorgani uchun LLM da ishlatilmaydi	Classic chuqur tarmoqlar
GELU	$x * \Phi(x)$	Yaxshi. Transformer asosi.	GPT-3, BERT
SwiGLU	$\text{Swish}(xW) \odot (xV)$	Best! Eng tez va aqilli	LLaMA 3, Gemini, Mistral
GEGLU	$\text{GELU}(xW) \odot (xV)$	Yaxshi. SwiGLU ga uxshaydi lekin sekinroq	PaLM, T5

Qachon normalization qilish kerak? 2 hil yuli bor:

1. Pre-norm

Input → Tokenization → Normalization → Attention/FNN (parallel) → Activation F

2. Post-norm

Input → Tokenization → Attention/FNN (parallel) → Normalization → Activation F

Hozir frontier modellar barchasi pre-norm yulini ishlatishadi. Va bu modern yul hisoblanadi

Transformer Architecture: Token & Vocabulary

Tokenization: suzlarni kichik qismlarga ajratish jarayon

1-step: tokenization. Gapni uqib qismlarga ajratadi. Yani tokenlarga

2-step: vocabulary. Ajratilgan bulaklarni uzining ichki listidan qidiradi. Ularning ID raqamlarini qaytaradi.

Vocabulary – suz bilan istamaydi a har bir suzni uzini unique ID si bilan qidiradi

Transformer Architecture: Architecture Variants

Architecture variants: modellar juda uzun textlar bilan ishlaganda, GPU ning memory si cache bilan tulib ketadi. shu muammoni yechish uchun ishlatamiz.

GPU out of memory bulib **modelimiz uchib qoladi** agar variantlardan foydalanmasak

Variants:

- **MHA** (Multi-head attention): eski classic usul. har bir query dagi savolning manosini (grammar, history tomonlari) topish uchun har bir qismga alohida key va value beradi
Uta ogir va sekin

- **MQA** (Multi-query attention): barcha qismga bitta key & value beradi. **uta yengil va tez**

- **GQA** (Grouped-query attention): qismlarni malum bir grouplarga bulib va har bir group uchun alohida key va value beriladi. **Golden standard**